



Universidad Nacional Experimental de Guayana

Coordinación General de Pregrado

Vicerrectorado Académico

Proyecto de carrera: Ingeniería en Informática

Asignatura: Ingeniería del Software II

Fundamentos de la Gestión de Proyectos y Productos de Software

Integrantes	C.i
Arnaldo Perdomo	30.791.551
Daniel Reyna	29.989.066
Jhoanny Maita	30.694.732
Yoryelis Ocando	31.074.651

Puerto Ordaz, 02 de Septiembre del 2026

Fundamentos de la Gestión de Proyectos y Productos de Software

1. Ejercicio– Resuelto por: Yoryelis Ocando

Traducción de Necesidades a Requerimientos y Actores Contexto: Un hospital público necesita reducir el tiempo de espera de los pacientes en urgencias. Instrucciones: Identifique al menos 4 actores (stakeholders) clave en este escenario. Para cada actor, redacte una Necesidad" (el problema encontrado) y traduzca a un Requerimiento específico del software (funcional o no funcional). ¿Cómo podría la IA o el prototipado ayudar a validar estos requisitos antes de programar?

Los actores o stakeholder son personas que se ven afectadas por el proyecto o tienen algo que aportar en él. Bajo esta visión, se identificaron los siguientes 4 actores claves, con sus respectivas necesidades operativas (El problema a resolver) y la traducción a requerimientos del sistema (las capacidades, restricciones que debe tener el software para resolver el problema planteado).

Este ejercicio se fundamenta en los siguientes estándares y marcos de referencia:

ISO/IEC/IEEE 29148 (Ingeniería de Requisitos): guía el proceso de investigación, recolección y redacción de requerimientos, garantizando que sean claros, verificables y factibles para resolver el problema hospitalario. SWEBOK V4 (IEEE): aporta la base metodológica para transformar las necesidades de los usuarios en especificaciones técnicas de software. PMBOK 7 (PMI): aplica los principios de Interesados (Stakeholders), al involucrar a los actores clave del hospital, y de Entrega de Valor (Value), asegurando que el sistema reduzca tiempos de espera reales (outcomes) y no se limite a la simple entrega de código (outputs).

- **Paciente:**

Necesidad: No sabe cuánto tiempo falta para ser atendido, empeorando el estado en el que se encuentra provocando complicaciones adicionales o incluso la muerte.

Requerimiento (funcional): El sistema debe mostrar en una pantalla pública o aplicación el tiempo estimado de espera y la posición en cola del paciente, actualizado en tiempo real.

- **Enfermería:**

Necesidad: Debe clasificar la gravedad de cada paciente, pero el registro manual en papel es lento y propenso a errores, retrasando la priorización de casos críticos.

Requerimiento (funcional): El sistema debe permitir registrar y clasificar a los pacientes según la gravedad de su condición en menos de 2 minutos, con campos predefinidos por nivel de urgencia, y reordenar automáticamente la cola según prioridad.

- **Médicos**

Necesidad: No tienen el historial clínico del paciente al momento de atenderlo, lo que le obliga a repetir preguntas o esperar papeleo, alargando la consulta.

Requerimiento (no funcional): El sistema debe integrarse con el historial clínico electrónico existente y presentar la información relevante del paciente en menos de 3 segundos de tiempo de carga, garantizando disponibilidad durante horario de urgencias.

- **Dirección del hospital**

Necesidad: No cuenta con datos objetivos sobre cuellos de botella (¿el retraso está en triaje, en diagnóstico, en camas disponibles?), por lo que no puede tomar decisiones informadas de gestión de recursos.

- **Requerimiento (no funcional):** El sistema debe generar reportes y dashboards con métricas de tiempo por etapa del proceso, exportables y con trazabilidad para auditoría.

El prototipado rápido (Figma, Balsamiq o canva) permite mostrarle a cada actor una representación tangible de la solución antes de escribir código, para confirmar que el requerimiento realmente resuelve su necesidad y no solo lo que el equipo cree que necesita.

La IA Generativa aporta un valor adicional: puede analizar grandes volúmenes de datos históricos de flujo de pacientes para sugerir requerimientos no funcionales (por ejemplo, picos de demanda por horario) o detectar patrones de uso que no se habían verbalizado explícitamente. Esto reduce la dependencia exclusiva de entrevistas y encuestas. Esto ya se ha implementado en el sistema de salud de países como Francia, Portugal, Hungría, Suecia y Países Bajos y a nivel de la Unión Europea la mayoría de los países miembros ha adoptado algún sistema de este tipo.

2. Ejercicio– Resuelto por: Daniel Reyna

Una startup decide desarrollar una app de delivery. El gerente técnico impone el uso de una arquitectura de microservicios en Kubernetes (Tecnología), pero el equipo solo sabe programar en PHP monolítico (Personas). No definieron un proceso de gestión de cambios (Proceso) y el cliente pide agregar una función de "pagos con criptomonedas" a mitad del desarrollo sin ajustar el presupuesto (Producto/Alcance).

- **Análisis:** Para poder visualizar lo que ocurre con lo que se plantea debemos de entender lo que Pressman establece sobre las 4 dimensiones, menciona que todo proyecto de software se sostiene sobre 4 pilares

fundamentales que deben de estar alineados, los cuales son: Persona, Proceso, Producto y Tecnología. Cualquier desbalance entre alguno de estos pilares garantiza el fracaso absoluto del proyecto.

Ahora bien, sabiendo lo que plantea Pressman, podemos analizar el desbalance en el caso que se plantea.

- **Personas:** Es considerado el elemento más importante, por definición son el capital humano, el equipo de desarrollo y su talento, por lo que Pressman enfatiza que las personas son el recurso primario más importante. Para el caso planteado, hay un desbalance severo, ya que el equipo solo sabe programar en PHP monolítico, siendo esta su zona de confort. El problema con esto es que para la tecnología impuesta se requieren conocimientos y habilidades en contenedores, orquestación, comunicación entre servicios, lenguajes como Go, Java o Node.js, Forzar al equipo a trabajar con tecnologías que no domina representa un desafío que puede llevar a la frustración, baja productividad y alta probabilidad de errores.
- **Proceso:** Es el conjunto de actividades y tareas que guían el desarrollo del software. Para el caso el mismo enunciado indica que “No definieron un proceso de gestión de cambios”, esto es un gran desbalance, sin un proceso, el equipo no tiene una guía clara de como integrar nuevas funcionalidades, como priorizar ni como documentar los avances. El problema con esto radica en que cualquier nueva solicitud del cliente (como la de criptomonedas) se convierte en un caos, no hay ninguna guía de cómo proceder.
- **Producto:** Se define como el software a entregar, este debe de cumplir con los requisitos solicitados. Para el caso planteado, sufre claramente de un caso de alcance no controlado, el cliente pide una función compleja (pagos con criptomonedas) a mitad del desarrollo, sin ajustar presupuesto. El problema radica en que agregar una nueva funcionalidad de esta magnitud sin un análisis de impacto, sin reestimar el esfuerzo y sin renegociar el presupuesto lleva fácilmente al desastre. El producto se aleja de su objetivo inicial y la dificultad aumenta demasiado.
- **Tecnología:** Se define como el conjunto de herramientas, lenguajes y plataformas que se utilizan para la construcción del software. Para este caso, el gerente técnico impone una arquitectura de microservicios en Kubernetes. Esta decisión tecnológica si bien es moderna y escalable, es muy inapropiada para el contexto. El problema radica en que la decisión es “Tod down” y no considera la capacidad real del equipo, la curva de aprendizaje es demasiado alta y el riesgo de fracaso es igual de alto.

Habiendo analizado las 4 dimensiones, se puede concluir que el proyecto tiene un desbalance total. La Tecnología es demasiado avanzada para las personas. La falta de Proceso descontrola el Producto. Este escenario es un

ejemplo clásico de cómo ignorar el factor humano y la realidad del equipo conduce al fracaso.

Teniendo en cuenta el desbalance que hay en el proyecto, también cabe destacar los 3 errores clásicos que logre identificar en este escenario. Para este análisis me base en el CHAOS Report del Standish Group y en la literatura clásica de Brooks.

- El primer error es: La falta de participación y alineación del usuario y el equipo

En este error el gerente técnico impone el uso de una nueva tecnología sin consultar al equipo. Es una decisión unilateral que no toma en cuenta las capacidades del equipo. Esto genera desmotivación y una desconexión entre la estrategia y la ejecución.

- El segundo error es: Scope Creep (alcance no controlado):

En este error el cliente pide agregar “pagos con criptomonedas” a mitad del proyecto sin ajustar el presupuesto. Esto entra en la definición de “Scope Creep”. No hay un proceso para evaluar el impacto de este cambio, lo que lleva a sobre costos, retrasos y un producto final que no cumple con las expectativas.

- El tercer error es: Ignorar la deuda técnica y la complejidad.

En este error se pretende obligar al equipo que solo maneja PHP a desarrollar microservicios en Kubernetes, esto es una receta de como crear una deuda técnica masiva, el código sería de baja calidad, tendrá un montón de parches y será muy difícil de mantener y escalar a futuro.

Con todo esto presente, mi propuesta de mitigación sería la siguiente:

Para poder mitigar la situación, principalmente debo de entender que mi rol no es de gerente de proyectos, sino de un gerente servicial, cuyo enfoque no es imponer, sino servir al equipo, eliminar los impedimentos y facilitar el éxito.

Mi propuesta para mitigar la situación involucra 4 acciones fundamentales que deben de ser ejecutadas teniendo en cuenta el rol establecido, las acciones a seguir serían las siguientes:

1. Acción: Diagnosticar y alinear.

Para esto necesito convocar a una reunión de emergencia con el gerente técnico y el equipo, esta reunión no está buscando echar culpa ni nada similar, sino más bien para escuchar los puntos de vista de cada parte, realizar preguntas como “¿Cuáles son los principales obstáculos que ve el equipo en esta arquitectura?”, siempre priorizando la empatía y la escucha activa, para entender el problema de raíz y no solo los síntomas.

2. Acción: Replanificar la estrategia técnica.

Esto para poder definir un plan de transición y no un cambio radical. La idea sería desglosar en cortos, medianos y largos plazos las actividades a realizar. Primeramente, a corto plazo se pediría desarrollar una primera versión de la app con la tecnología que el equipo domina, esto permitiría entregar valor más rápido. A mediano plazo mientras se desarrolla la primera versión de la app, se capacitará al equipo para las tecnologías que necesitan. Por último, a largo plazo, se migraría poco a poco a microservicios, modulo por modulo, de acuerdo al avance del equipo.

3. Acción: Implementar un proceso de gestión de cambios.

Esto permitirá manejar las solicitudes del cliente. Primeramente, se crearía una lista donde se definan las prioridades respecto a las funcionalidades. Cada nueva petición deberá de ser analizada por el equipo, esto para estimar el esfuerzo y el impacto técnico. Y por último presentar al cliente un análisis de impacto. Esto para definir las prioridades del cliente, puede ser planteando una pregunta así: “La función de criptomonedas que nos solicita nos tomaría X semanas extras y costaría más, ¿Cómo prefiere proceder? ¿La priorizamos sobre otras funciones o la dejamos para una segunda versión?”. De esta forma el cliente toma parte del proyecto y define de mejor forma sus prioridades.

4. Acción: Proteger al equipo y fomentar la autoorganización

Esto con la intención de empoderar al equipo para que tome decisiones técnicas dentro de un marco claro. En lugar de que el gerente técnico imponga la arquitectura, que el quipo, con la guía del líder, decidan la mejor ruta técnica para cumplir los objetivos.

Con estas acciones considero que se puede mitigar la situación, logrando que el proyecto se pueda llevar a cabo de una manera más suave y con la menor cantidad de impedimentos, así como también creando una estructura clara y limpia para trabajar.

3. Ejercicio - Resuelto por: Jhoanny Maita

Enfoque de gestión: PMBOK 6 vs PMBOK 7

El estándar del Project Management Institute (PMI) experimentó un cambio paradigmático entre su sexta y séptima edición:

Criterio	PMBOK 6 (Basado en Procesos)	PMBOK 7 (Basado en Principios)
Estructura	49 procesos distribuidos en 5 Grupos de Procesos y 10 Áreas de Conocimiento.	12 Principios orientadores y 8 Dominios de Desempeño.

Enfoque nuclear	Cumplimiento del plan predictivo y control del "Triángulo de Hierro" (alcance, tiempo, costo).	Sistema de entrega continua de valor y beneficios de negocio (outcomes).
Gobernanza	Prescriptiva: define entradas, herramientas/técnicas y salidas (ITTOs) estructuradas.	Holística y adaptable: aplica Tailoring (adaptación al contexto: predictivo, híbrido o ágil).
Criterio de éxito	Producción conforme de entregables técnicos (outputs) según la línea base inicial.	Impacto real en los interesados y resolución efectiva de problemas de negocio.

Impacto en la gestión de requisitos de software

Según el PMBOK 6, la ingeniería de requisitos se maneja a través de procesos bastante rígidos, como Recopilar Requisitos, Definir el Alcance y Crear la EDT/WBS. La idea es establecer una línea base del alcance que se congele mediante documentos formales, como una SRS o SyRS, siguiendo la norma ISO/IEC/IEEE 29148. Cualquier cambio que surja durante el ciclo de vida se considera una desviación o amenaza (Scope Creep), lo que lleva a la necesidad de estrictos comités de control de cambios (CCB).

Por otro lado, el PMBOK 7 ofrece una nueva perspectiva, viendo los requisitos como hipótesis de solución dentro de un ecosistema complejo. Se basa en principios como Valor, Interesados, Pensamiento Sistémico, Adaptabilidad y Resiliencia, así como en Habilitar el Cambio. Así, los requisitos dejan de ser un contrato inamovible y se gestionan como elementos dinámicos, como Product Backlogs y User Stories. Se pone un fuerte énfasis en el descubrimiento continuo (Product Discovery) y en ciclos cortos de retroalimentación (feedback loops), lo que optimiza el impacto (outcome) en lugar de centrarse en la rigidez de la documentación (output).

Ejemplos explicativos

- Ejemplo 1: Descubrimiento de fricción operativa en pasarelas de pago

Bajo PMBOK 6 (Procesos): En la fase de planificación, se da luz verde a un formulario de pago web que requiere autenticación por SMS y tiene 8 campos obligatorios. Sin embargo, a mitad del desarrollo, las métricas y pruebas iniciales indican que muchos usuarios abandonan el proceso porque resulta complicado. Como la especificación técnica ya fue aprobada, el enfoque

tradicional del PM se niega a simplificar el formulario o exige abrir una solicitud formal de cambio para mantener el cronograma y el costo acordados. El resultado es un producto que sigue el plan al pie de la letra, pero con una baja tasa de conversión y descontento entre los usuarios.

Bajo PMBOK 7 (Principios): Siguiendo el principio de Enfocarse en el Valor y de Adaptabilidad, el equipo se da cuenta de la baja adopción y decide reemplazar el flujo por una autenticación biométrica de un solo clic. En lugar de aferrarse a una especificación que ya no sirve, ajusta el Product Backlog en la siguiente iteración. Aquí, el éxito no se mide por seguir la documentación original, sino por el resultado: la retención de usuarios y el aumento en el volumen de transacciones efectivas.

- **Ejemplo 2: Cambio normativo o regulatorio durante la fase de pruebas**

Bajo PMBOK 6 (Procesos): Mientras se desarrolla una plataforma de comercio electrónico, entra en vigor una nueva ley de protección de datos que exige un consentimiento más detallado para las cookies. Con el enfoque de procesos, esto se considera una interrupción crítica que detiene el despliegue; se debe documentar el impacto en el presupuesto, pasar por la junta de cambios y recalcular la ruta crítica, lo que retrasa la salida a producción de manera reactiva.

Bajo PMBOK 7 (Principios): Aplicando el Pensamiento Sistémico reconociendo cómo el entorno legal afecta al subsistema de software, y Habilitar el Cambio, el equipo integra la nueva restricción en su arquitectura modular sin caer en la burocracia. Reorganiza las prioridades del backlog colaborando con el área legal para asegurar que la conformidad regulatoria se implemente de manera efectiva.

4. Ejercicio– Resuelto por: Arnaldo Perdomo

En la ingeniería de software moderna, el mayor riesgo no es que el código no compile, sino que el software se construya perfectamente, pero nadie lo use. Discuta esta afirmación relacionándola con la importancia de la ingeniería de requisitos, la identificación de actores y la crisis del software.

- **Análisis**

Se dice que el mayor riesgo en un proyecto de software no es que el código no compile, sino que se construya perfectamente y nadie lo use. Esta frase cambia por completo la forma en que solemos entender el fracaso en ingeniería de software. Durante años pensamos en el fracaso en términos técnicos: errores de compilación, sistemas caídos, entregas atrasadas. Pero hay algo más incómodo detrás de esa afirmación: un proyecto puede ejecutarse de forma impecable como tener un código limpio, una arquitectura sólida, entregas a tiempo y aun así fracasar por completo, simplemente porque a nadie le importa el resultado.

- **De la crisis técnica a la crisis de valor**

La llamada “Crisis del Software”, que se nombró en la conferencia de la OTAN, hablaba de proyectos que se salían de presupuesto o se atrasaban realizando entregas de baja calidad. En el fondo era una crisis de capacidad: no sabíamos construir software de forma confiable. Con el tiempo, primero con Cascada, luego con Ágil y después con DevOps, la industria fue resolviendo ese problema. Hoy en día podemos desplegar código funcional en minutos y con bastante confiabilidad. Que el código no compile ya casi no es la causa de fracaso que solía ser.

Pero la crisis no desapareció, solo cambió de forma. Dejamos de fallar en construir software y empezamos a fallar en decidir qué construir. A esto se le conoce como el “síndrome de la fábrica de funcionalidades”: equipos muy productivos que entregan sprint tras sprint, sin detenerse nunca a preguntarse si lo que están construyendo resuelve un problema real. Un proyecto puede cumplir el Triángulo de Hierro a la perfección, y aun así ser un fracaso total como producto, porque ese triángulo mide qué tan bien ejecutamos, no si lo que hicimos sirve de algo.

Ahí aparece una distinción importante: gestionar un proyecto no es lo mismo que gestionar un producto. La gestión de proyectos se conforma con el output, es decir, con entregar lo que decía la especificación. La gestión de producto exige el outcome: el impacto real, medido en cosas como adopción, retención o valor de negocio. El riesgo del que habla el enunciado solo se hace visible cuando dejamos de preguntarnos “¿lo entregamos?” y empezamos a preguntarnos “¿alguien lo está usando?”.

- **El rol de la ingeniería de requisitos**

Este riesgo de construir algo inútil casi siempre nace de una ingeniería de requisitos mal hecha. Hay una diferencia clave entre necesidad, es decir, el problema real que tiene un stakeholder y la forma en que esa necesidad se traduce en una especificación de software. Cuando esa traducción se hace mal, porque nadie elicitó bien la información, porque se asumió en vez de preguntar, o porque se documentó literalmente lo que el cliente dijo y no lo que en realidad necesitaba, el equipo puede terminar ejecutando con total precisión un plan que apunta al objetivo equivocado.

La Curva de Boehm nos enseña que un error de requisitos detectado en la etapa de mantenimiento puede costar hasta cien veces más que si se detecta desde la elicitación. Pero el caso que plantea este ejercicio es, en realidad, todavía peor: no se trata de un error que se arregla con retrabajo, sino de un fracaso terminal. Si nadie usa el software, no hay ciclo de mantenimiento que valga la pena, simplemente se perdió toda la inversión. Por eso estándares como el ISO/IEC/IEEE 29148 insisten en que un requerimiento bien redactado debe ser verificable, necesario y trazable a una necesidad real; no basta con que sea técnicamente implementable.

- **El rol de la identificación de actores**

El otro origen de este riesgo está en identificar mal a los actores del proyecto. Algo que suele pasarse por alto es que no todos los actores quieren que el proyecto se concrete: también hay quienes prefieren que no se concrete, o a quienes simplemente les da igual. Si el equipo solo escucha al patrocinador que paga o al gerente que aprueba el presupuesto, y nunca llega hasta el usuario final que va a operar el sistema día a día, el software terminará resolviendo el problema que el patrocinador cree que existe, no el que el usuario realmente enfrenta.

Este es, de hecho, el primer error clásico que señala el CHAOS Report: la falta de participación del usuario, construir lo que el cliente cree que quiere y no lo que en verdad necesita. Un equipo puede tener procesos ágiles impecables, entregar cada dos semanas y tener 90% de cobertura de pruebas, y aun así fallar en esto, porque ser ágil ejecutando no compensa haber identificado mal para quién se está construyendo.